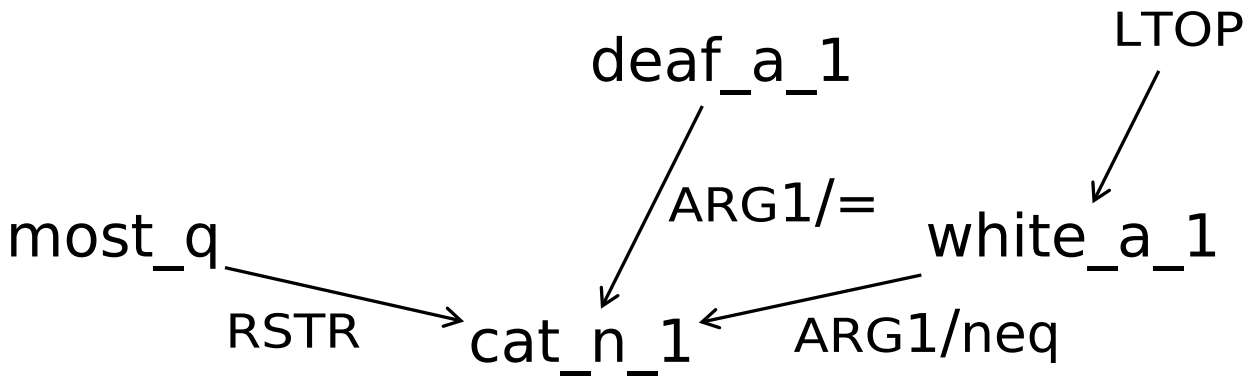
Semantic Functions

Guy Emerson and Ann Copestake

Delph-in 2015

What is Lexical Semantics?

"Most deaf cats are white"

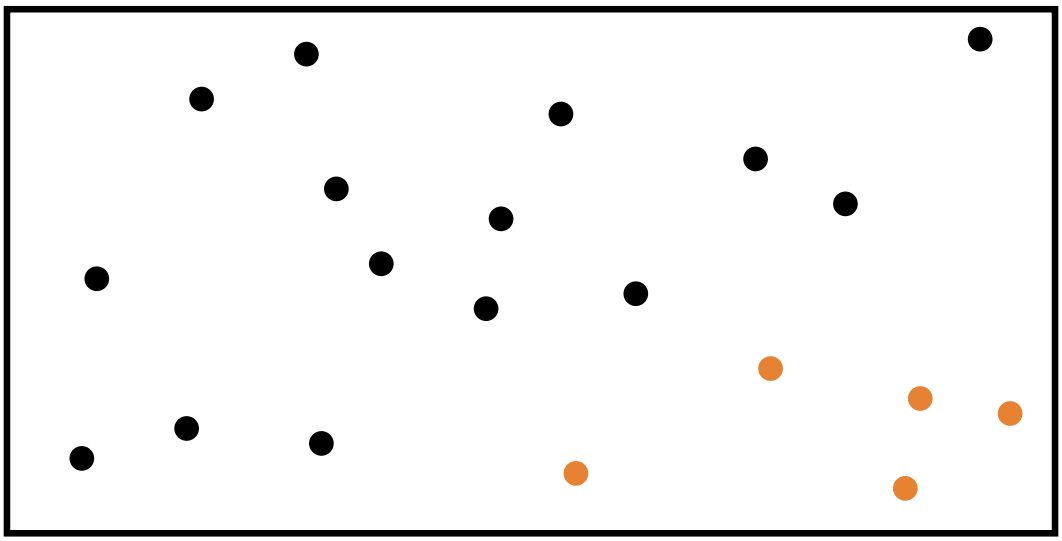


We have a well-developed framework for representing semantics, but how are we supposed to interpret the elementary predications? If we want to approach the task with machine learning, what kind of object are we trying to learn?

Overview

- Review: model theory, vector spaces
- Proposal: *semantic functions*
- Theoretical benefits
- Learning and inference

Model Theory



dog′

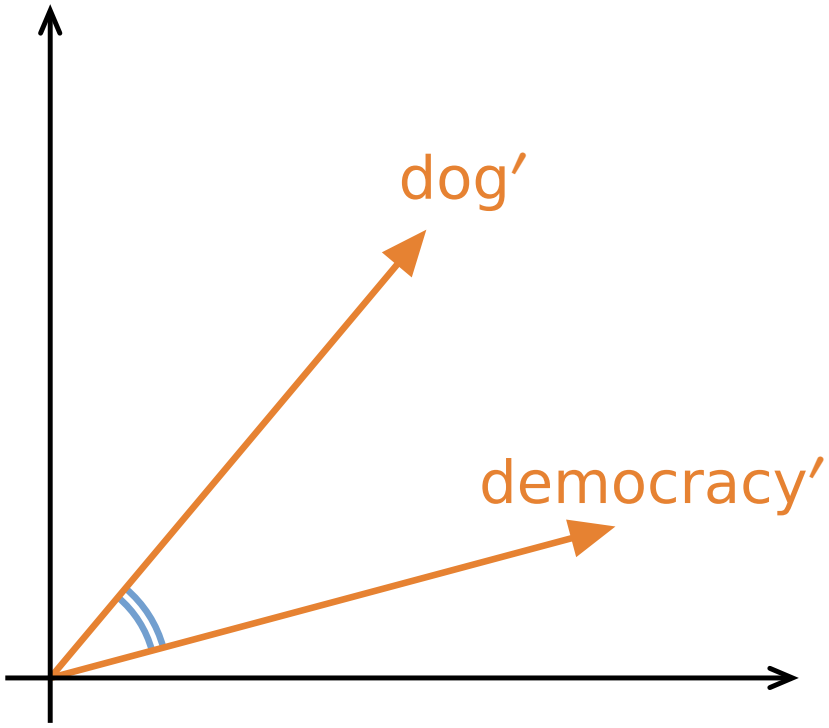
In model theory, predicates are interpreted in a model structure, which is a set of individuals. The *denotation* of a predicate is simply a subset. This theory has been extensively explored.

Model Theory: Difficulties

- Gradedness
- Polysemy
- Context dependence
- Non-denotational structure
- Machine learning

Sets do not naturally model various linguistic phenomena.

Vector Spaces



In a vector space model, we model meanings as points in a (high-dimensional) vector space. Various methods have been established for constructing such vectors, and some calculations are simple, such measuring similarity as the cosine of an angle.

Vector Spaces: Difficulties

- Gradedness
- Polysemy
- Context dependence
- Inference
- Compositionality
- Grounding

Vectors do not naturally model various linguistic phenomena.

Definition of a Semantic Function

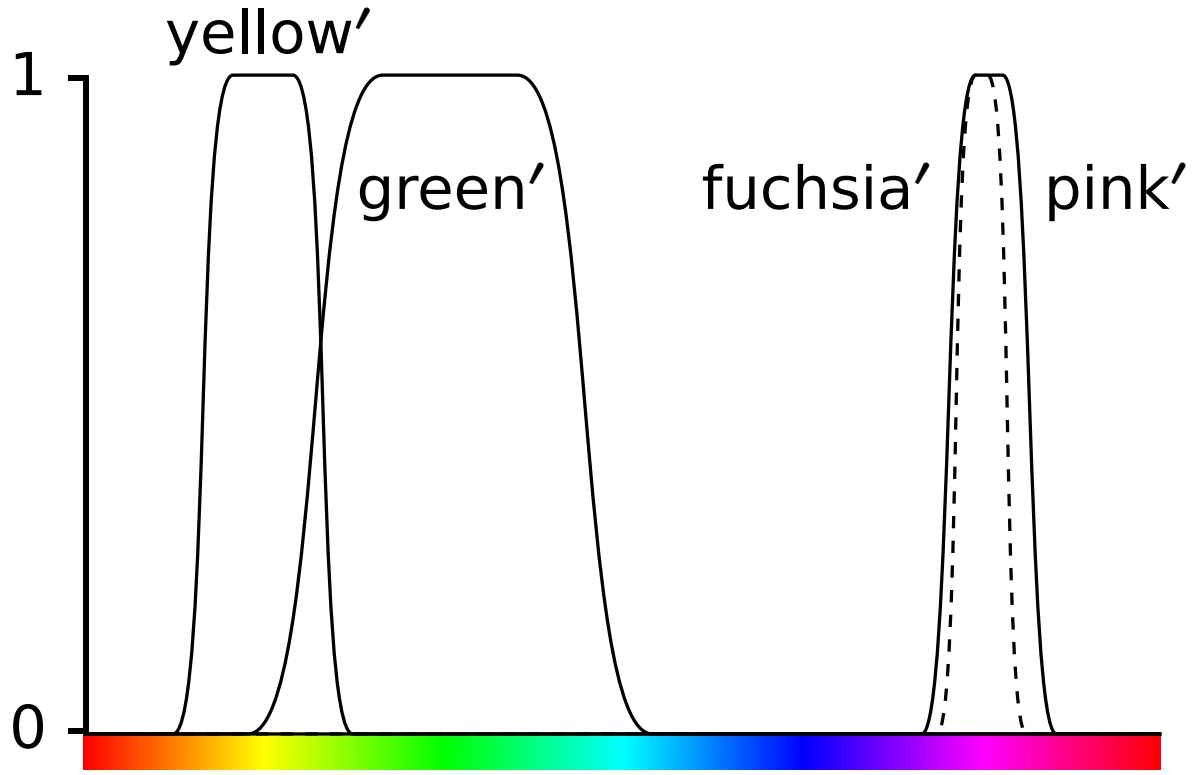
$$f : S \rightarrow [0, 1]$$

space of possible entities

values between 0 and 1

Assume we have some kind of semantic space, where each point represents a possible entity. This is *not* a model structure, as we are considering all possible entities, not those in some world. Furthermore, this space can be structured, such as a continuous high-dimensional space, where each dimension represents some semantic feature. We can view the meaning of a predicate as a function, mapping points in the semantic space to values expressing how applicable the predicate is for that point. The value 1 would indicate that the predicate is definitely applicable, and 0 that it is definitely not. An intermediate value like 0.5 would indicate a borderline case.

Example: Colour Terms



Considering hues, we have a simple one-dimensional space. Each colour term’s function takes the value 1 for hues it definitely describes, and 0 for the hues it definitely doesn’t. For borderline hues, the function takes intermediate values. For example, for yellowish greens, we have intermediate values for both yellow and green. Hyponymy between terms can be easily captured by one function falling beneath another, as shown for fuchsia and pink.

Benefits

- Gradedness – intermediate values
- Vagueness – region of *S*
- Grounding – choice of *S*
- Hyponymy – function beneath another
- Polysemy – non-protypical function

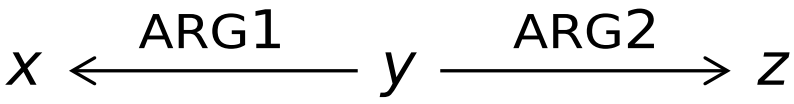
For time reasons, I can’t go into detail here, but I believe this framework can capture these phenomena more naturally than model theory or vector spaces.

Integration with MRS



In the following section, I will explain how to connect the semantic functions to MRS, for declaratives, neglecting quantifiers.

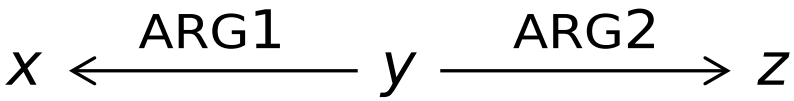
Integration with MRS



dog′(x) chase′(y) cat′(z)

We can tease apart the characteristic variables from the elementary predications in something between DMRS and RMRS.

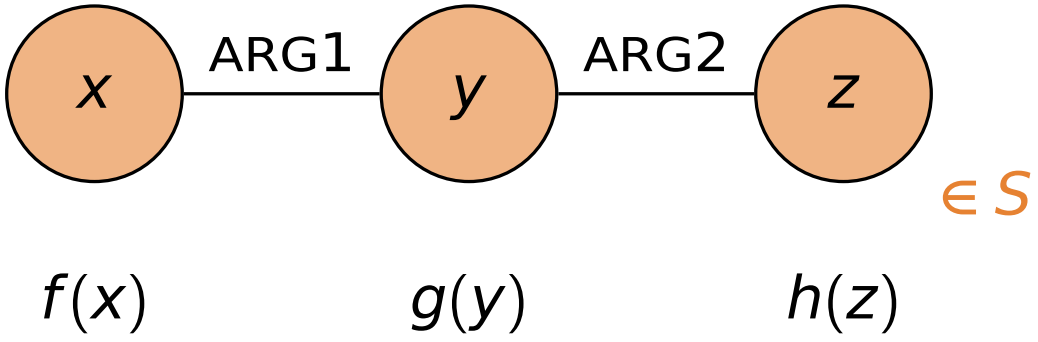
Integration with MRS



$f(x)$ $g(y)$ $h(z)$

I'll write f, g, and h for generic predicates.

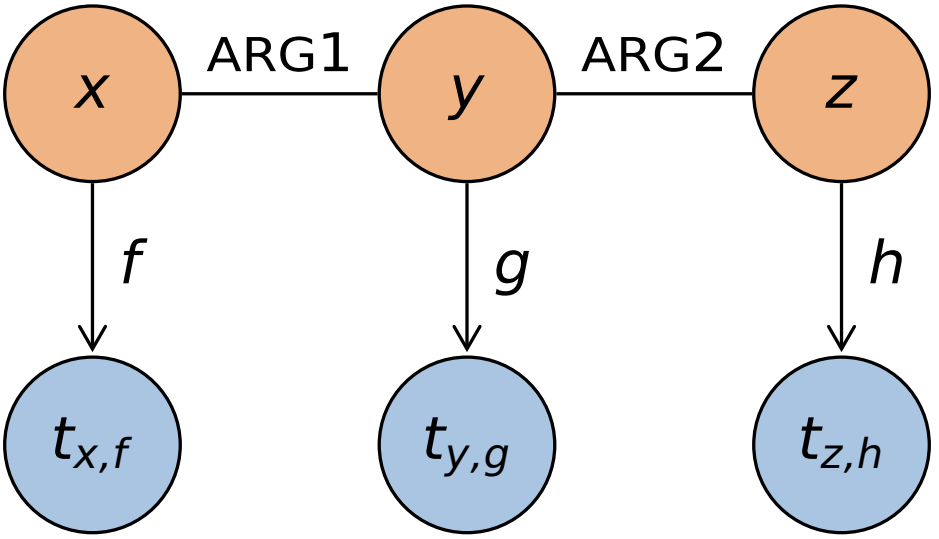
Integration with MRS



$f(x)$ $g(y)$ $h(z)$

We can model the entities as points in the semantic space S. Further, to represent our uncertainty, we can model them as random variables taking values in S. We have a joint distribution for all of the variables, and the ARG links introduce statistical dependence between them. For example, we should have a higher probabililty for x, y, and z jointly lying in the parts of the space for dog, chase, and cat; or person, watch, and film; and a lower probability for dog, chase, and democracy; or film, chase, and person. Formally, we can represent this joint distribution using a Markov Network. Each tuple of possible entities represents a possible situation.

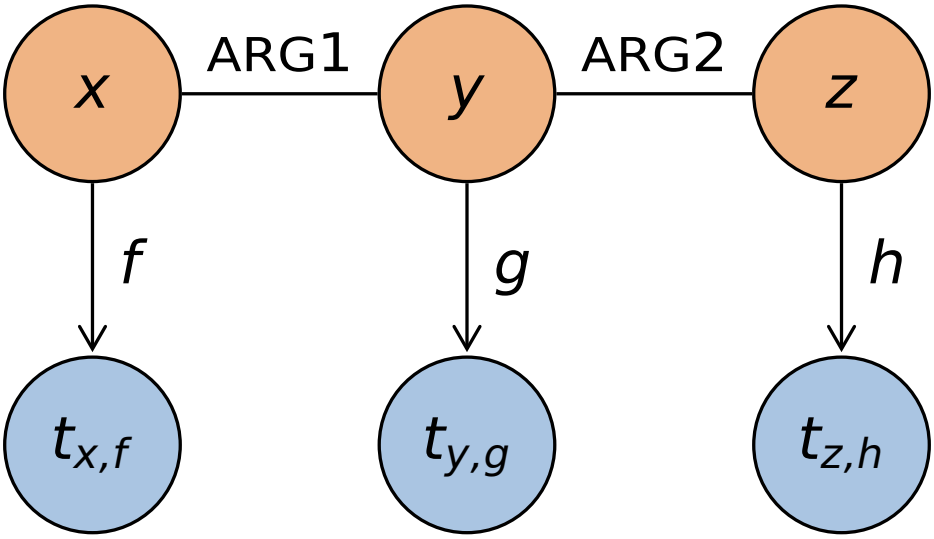
Integration with MRS



$$P(t_{x,f} = \top | x) = f(x)$$

We can model the predicates as semantic functions acting on the variables. If the MRS graph accurately describes the given situation, all of the semantic functions should take values close to 1. To keep everything within a probabilistic framework, we can interpret the values of the functions as probabilities of a binary-valued random variable being true (and 1 minus the value as the probability of being false). For example, $t_{x,f}$ represents whether or not the predicate f is applicable to the entity x.

Inference



$$P(x, y, z \mid t_{x,f}, t_{y,g}, t_{z,h}) \propto P(x, y, z) f(x) g(y) h(z)$$

situational knowledge — linguistic knowledge

This framework allows inference, in a much more natural way than for vectors. However, unlike many ‘traditional’ linguistic theories, this is Bayesian inference, not logical inference. For example, we can apply Bayes Theorem to infer features of the entities, given the truth of certain predicates. The posterior distribution for the entities then depends on the prior distribution, which represents our background knowledge of the situation, and the semantic functions, which represents our linguistic knowledge.

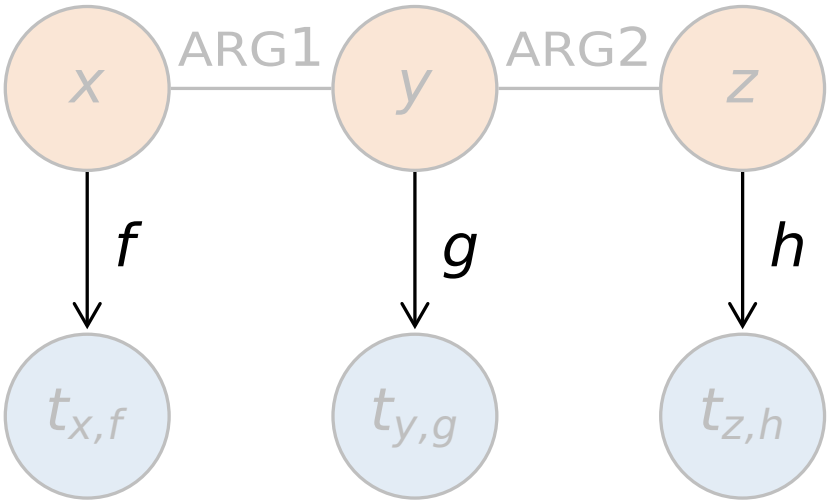
Context Dependence

a large hamster

a small elephant

Although we want to say that ‘large’ and ‘small’ have some core meaning, their interpretation in context will vary: for instance, a large hamster is smaller than a small elephant.

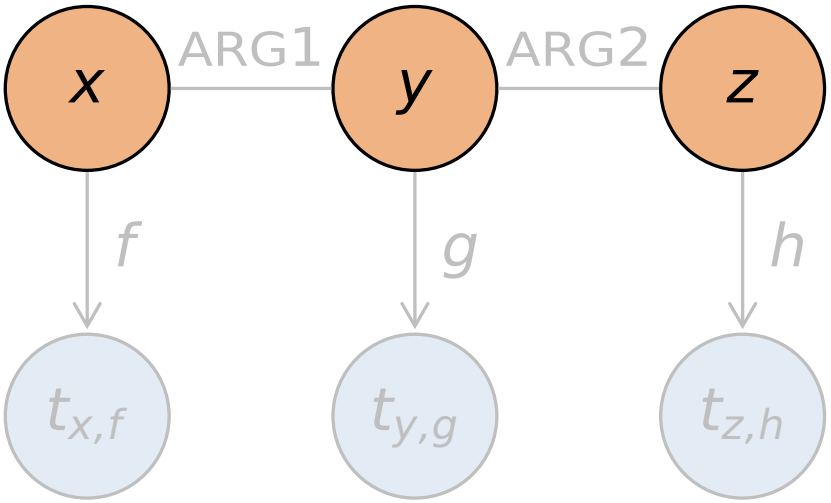
Context Dependence



Standing Meaning

The semantic functions do not change for each context.

Context Dependence

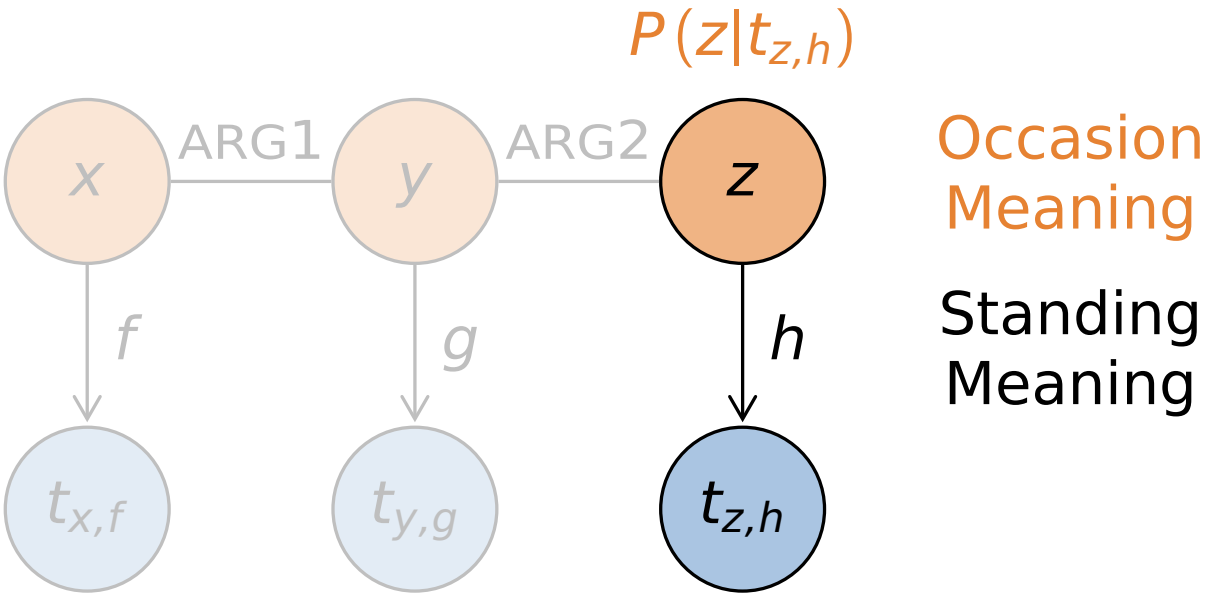


Occasion Meaning

Standing Meaning

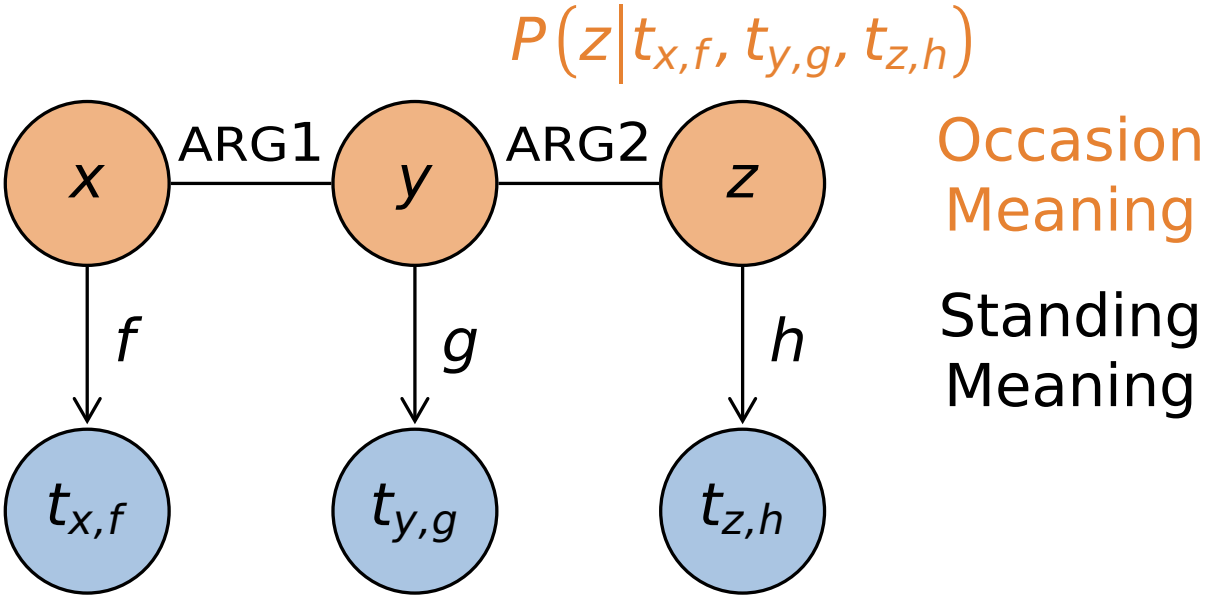
The entities are different in each context.

Context Dependence



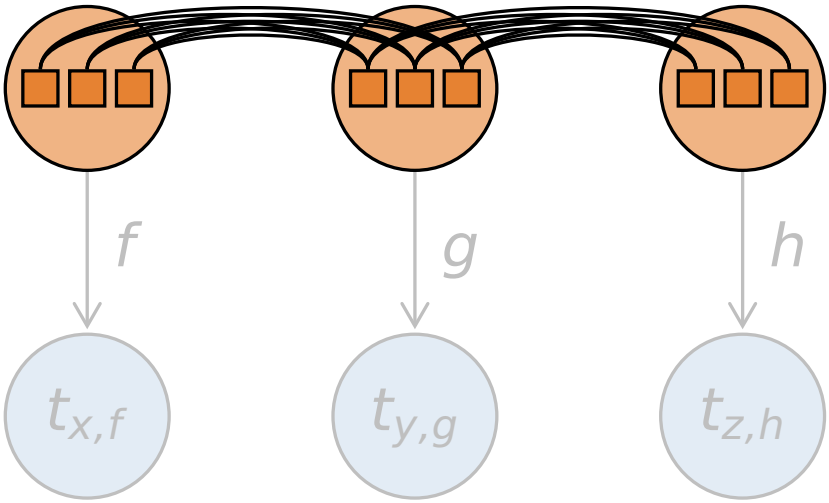
If we just consider one entity, we can infer the features it is likely to have, given that the predicate is true.

Context Dependence



If we now consider the entire DMRS, then information from all of the predicates can propagate to the entity. The posterior distribution for *z* then depends on the entire graph.

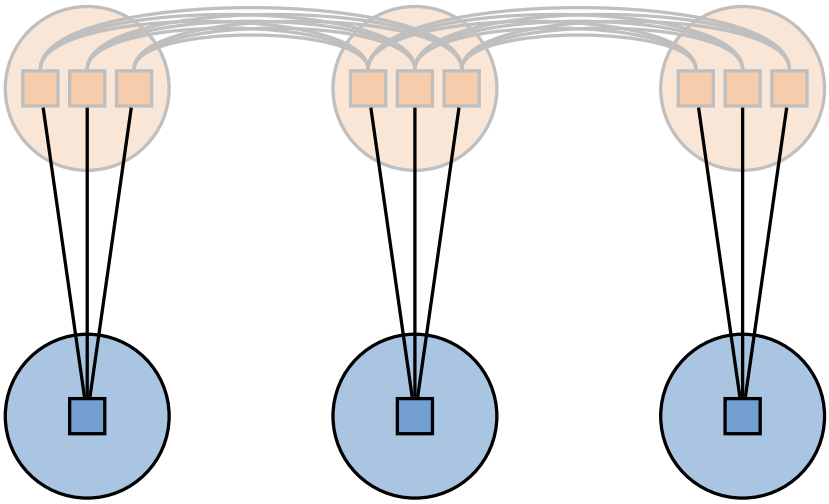
Implementation



Restricted Boltzmann Machine

The entities and the links can be modelled as a Restricted Boltzmann Machine. Each entity consists of an array of binary-valued components, each connected to all the values of adjacent entities. The probability of each component being on or off (1 or 0) depends on the values of all components it’s connected to.

Implementation

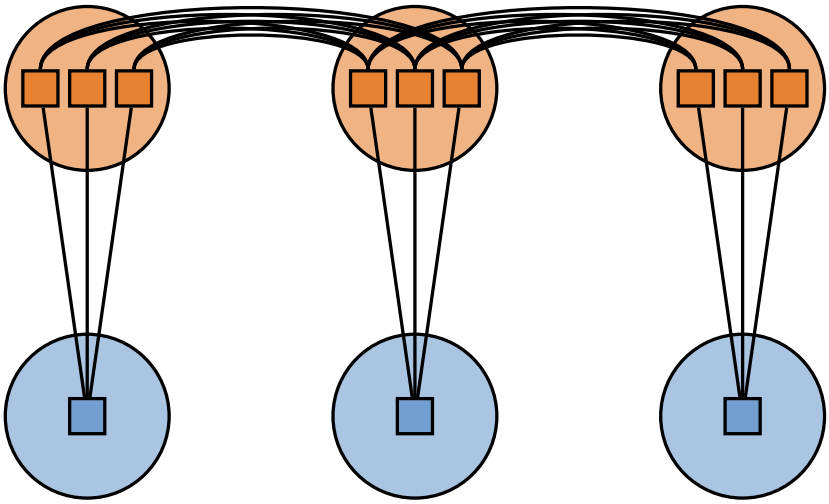


Restricted Boltzmann Machine

Feedforward Neural Network

A semantic functions can be modelled as a feedforward neural network. This takes an array as an input and outputs a value.

Implementation



Restricted Boltzmann Machine

Feedforward Neural Network

We might want to include further hidden layers, but this diagram broadly shows the overall architecture. We need to learn the weights, both in the RBM and for the semantic functions. If we are learning from a corpus of DMRSs (such as WikiWoods), the entities are not known, so we must marginalise them out during training.

Training Procedure

Need to update:

- Forward weights → Backpropagation
- Link weights → Contrastive divergence
- Latent variables → Gibbs sampling

Backpropagation is a standard technique for supervised training of neural networks – if we know the correct output, we can modify the weights to increase the chance of producing this output. Contrastive divergence is a standard technique for unsupervised training of RBMs – we modify the weights so that what the model generates looks more like the data. Gibbs sampling is a standard technique to deal with latent variables – for each variable, we update what it might be, conditioned on all other variables.

Conclusion

- Lexical semantic framework aiming to:
  - Build on formal semantics
  - Exploit machine learning
- Questions? Comments?